
DiffKV

Anchor + Low-Rank Differential KV-Cache Compression for Long-Context Inference on Commodity Unified-Memory Hardware

Technical Report

Generated July 5, 2026

A sparse KV-cache inference runtime that compresses each transformer KV block to an *anchor token*, a rank-16 joint SVD *delta*, and a set of *exact residual tokens*, paired with a bounded dense recency window and a fused, routed decode kernel that scores queries in low-rank space without ever decompressing the cache and attends the residuals exactly. The bounded store reaches 64k context where a dense cache exhausts an 8.6 GB Apple M3, and the residuals recover a buried passcode exactly at every context. Evaluated on Qwen2.5-1.5B (int4). All numbers in this report are measured; no value is fabricated.

Abstract

The key-value (KV) cache is the dominant memory cost of long-context transformer inference: it grows linearly with sequence length and, on a single commodity device, becomes the binding constraint long before compute does. We present DIFFKV, a KV-cache inference runtime that attacks this cost structurally rather than by eviction or quantization alone. DIFFKV partitions the cache into fixed-size blocks and represents each block three ways at once: an *anchor* token, a low-rank *differential* term (a rank-16 joint randomized SVD of the per-token deviations from the anchor, shared across keys and values), and a small set of *exact residual tokens*—the handful of tokens whose low-rank reconstruction error is largest, kept verbatim in fp16. The low-rank term captures a block’s shared structure cheaply; the residuals restore the low-energy outliers (a rare entity, a buried passcode) that pure low-rank compression discards. A sliding dense *recency window* keeps the most recent tokens uncompressed, and the whole store is a pre-allocated, fixed-size pool whose footprint is *bounded and context-independent* (≈ 0.68 GB across 28 layers), while a dense full-KV cache grows without bound. At decode, a single fused, compiled kernel routes the query to the most relevant blocks by an *exact* residual-key score, evaluates those blocks’ attention *directly in low-rank space*—never reconstructing dense keys or values—attends their exact residuals together with the recency window, and merges the two branches with a numerically stable log-sum-exp. An adaptive policy uses exact full-KV decode at short context and switches to the compressed kernel only once the cache is large enough to be the binding cost.

We implement DIFFKV on Apple’s MLX framework and evaluate it on Qwen2.5-1.5B-Instruct (int4) on an 8.6 GB Apple M3, with a needle-in-a-haystack protocol from 4k to 64k tokens. We report results honestly and separate the compressed sparse decode—the runtime’s actual contribution—from an exact full-KV decode used as an upper-bound ablation on the same store. DIFFKV’s compressed KV state is $\approx 2.85\times$ smaller than the dense cache at equal precision and remains bounded as context grows, letting the model process a 64k prompt on a device where the dense baseline exhausts memory by 32k—and the residual mechanism makes the compressed decode recover the buried passcode *exactly* at every tested context. These benefits come at measured costs we quantify rather than hide: the compressed decode is several times slower than dense decode (an implementation gap in the kernel dispatch, localized by the exact ablation), and the per-block SVD makes prefill the slowest phase. We characterize the central residual-budget trade-off—more exact residuals buy recall at the cost of compression—and design every figure and table so a future CUDA/Triton fused-decode implementation can be inserted without restructuring the evaluation. A full reproducibility appendix accompanies the report; all reported numbers are measured on the same machine, and no value is averaged across conflicting runs or fabricated.

Contents

1	Introduction	6
2	Background and Motivation	7
2.1	The KV cache and why it dominates	7
2.2	The design space	7
2.3	Unified memory and MLX	8
3	System Overview	8
4	Differential KV Compression	10
4.1	Notation	11
4.2	Anchor + differential representation	11
4.3	Normalized randomized truncated SVD	11
4.4	Exact residual tokens: restoring the outliers SVD discards	12
4.5	Storage budget and complexity	12
5	Runtime and Memory Architecture	13
5.1	Per-session state	13
5.2	Chunked, exact prefill with streaming compression	13
5.3	The prefill→decode boundary and the adaptive decode policy	13
5.4	Decode-step bookkeeping	14
5.5	Session lifecycle	14
6	Fused Sparse Decode Attention	15
6.1	Top- K block routing	15
6.2	Scoring and aggregating the low-rank term	16
6.3	Exact residual + recency branch	17
6.4	Stable two-branch merge	17
6.5	Compute profile	18
7	Implementation	18
8	Evaluation	19
8.1	Setup	19
8.2	E1 — KV-state footprint (the memory result)	20
8.3	E2 — Context reach	21
8.4	E3 — Prefill time	21
8.5	E4 — Decode throughput	21

8.6	E5 — Needle correctness	22
8.7	E6 — The residual-budget trade-off	22
8.8	Allocator peak and the combined view	23
9	Analysis and Discussion	23
9.1	What the compression actually buys	23
9.2	Why the allocator peak tells a different story	24
9.3	The throughput cost	25
9.4	Fidelity: restored by residuals, at a memory price	25
10	Limitations and Future Work	26
11	Conclusion	27
A	Reproducibility	29
B	Full Measured Results	30
C	Notation	31

List of Figures

1	DIFFKV active-runtime architecture. A serving layer drives the <code>MLXDiffKVWrapper</code> , whose patched attention routes <i>prefill</i> (exact attention over a native cache, with streaming capture and compression) and <i>decode</i> (ingest, flush, and fused compressed+dense attention). Both phases read and write a single per-session KV store: a bounded dense recency window plus a fixed compressed pool.	9
2	Compressing one 256-token block. The first token is the anchor; the remaining 255 tokens are represented as their deviations from it. Keys and values are stacked and jointly factorized by a rank-16 randomized truncated SVD, yielding a shared per-token coefficient matrix U and separate key/value bases V_K, V_V . Crucially, the $R=64$ tokens whose low-rank reconstruction error is largest are <i>additionally</i> retained as exact fp16 key/value pairs (residuals): SVD discards exactly the low-energy outliers that a buried needle token is, so the residual set restores verbatim recall (§8).	10
3	Per-session KV store (one of 28 layers). A fixed compressed pool of $M=256$ blocks (pre-allocated, zero-filled) plus a sliding dense recency window of capacity $W+B=768$ tokens. The pool’s footprint is independent of context length; the window is bounded. Overflowing the window compresses its oldest block into the pool.	14
4	Cache lifecycle. Chunked exact prefill streams compression into the pool; the prefill→decode boundary releases the prefill activations and drops the native cache; decode then ingests, compresses on overflow, and attends from the bounded store.	16
5	The decode path. A cheap router ranks compressed blocks by an exact query–key score over each block’s anchor and its top residual keys, and keeps the top $K=16$. Those blocks’ queries are scored <i>in low-rank coordinates</i> (never reconstructing dense keys); their exact residual tokens are concatenated with the dense recency window and attended exactly. The low-rank and exact branches are merged by a numerically stable log-sum-exp inside one <code>mx.compile</code> -d Metal graph.	17
6	KV-state footprint vs. context (all 28 layers). DIFFKV’s store is bounded and below the linearly growing dense cache; the gap widens toward the per-block $2.85\times$ bound as more tokens are compressed. Analytic bytes from the live store, residuals included; no estimation.	20
7	Decode throughput vs. context. Compressed decode is flat and slower; the exact ablation recovers dense-like throughput, localizing the gap to the kernel dispatch. A CUDA/Triton fused-decode series can be added here without restructuring (future work).	22
8	Prefill time vs. context (log scale). Exact attention plus streaming SVD and residual extraction; near-linear growth, slowest among the MLX configurations.	23
9	Recall vs. residual budget vs. memory at a fixed long context. Green/red markers are needle recovered/missed; the store grows with R while the compression ratio falls. Exact recall is bought with memory.	24
10	MLX allocator peak vs. context. Nearly identical across configurations—peak is weights- and prefill-dominated. Compression’s win is context reach and steady-state decode memory, not the global peak; we report it honestly rather than overstate a peak reduction.	26

11	Combined view: (a) KV-state footprint, (b) decode throughput, (c) prefill time, for DIFFKV compressed, the exact ablation, and the dense baseline.	27
----	--	----

List of Tables

1	Per-256-token-block storage budget (one layer, fp16). Derived from the model dimensions ($H_{kv}=2$, $d=128$) and configuration ($B=256$, $r=16$, $R=64$). The residual KV pairs dominate and account for the difference between the naive $10\times$ (low-rank only) and the true $2.85\times$	13
2	Main results: DIFFKV compressed decode (primary), exact decode (upper-bound ablation), and the dense baseline. All measured on the same Apple M3; greedy 128-token decode. The dense baseline is run to the point of memory exhaustion on this device.	21
3	Residual-budget trade-off (E6): recall, decode throughput, and store footprint as the exact-residual budget R varies, at a fixed long context.	25
4	Per-run detail: compressed decode (primary), exact decode (ablation), dense baseline.	31

1 Introduction

Autoregressive transformer inference caches the key and value projections of every past token so that each new token attends over history without recomputation. This KV cache is linear in sequence length, in the number of layers, and in the number of key/value heads. For a model the size of Qwen2.5-1.5B the per-token KV state is small, but it accumulates: at 64k tokens the full-precision cache alone is larger than the model weights, and on a shared 8 GB device it is the first resource to run out. The practical question this work addresses is narrow and concrete: *what does it take to run a long-context workload on the hardware people actually own*—a laptop with unified memory and no discrete GPU—rather than on a server with hundreds of gigabytes of HBM?

The standard answers each give up something. *Quantizing* the cache to 4 or 2 bits shrinks it by a constant factor but still grows linearly and eventually loses fidelity on the tokens that matter. *Evicting* tokens (keeping only “heavy hitters” or a sliding window) bounds the cache but discards information that a later query may need. *Low-rank latent* caches change the model architecture and require retraining. We pursue a different point in the design space, motivated directly by the structure of the cache: KV vectors within a local block are highly correlated, so each block is well approximated by a single representative token plus a low-rank correction—*except* for a few distinctive tokens whose detail a low-rank model discards but a later query may need verbatim. This is the central idea of DIFFKV: store an *anchor*, a *differential* low-rank term, and a small set of *exact residual tokens* per block, and never materialize the dense cache again—not even at attention time. The low-rank term gives the bounded memory; the residuals give back the exact recall that a pure low-rank scheme loses.

What DiffKV is. DIFFKV is a runtime, not a model change. It monkey-patches the attention of a stock quantized model and routes the two phases of inference differently. During *prefill*, it runs exact attention while streaming the produced KV into a compressed store: every 256-token block is reduced to an anchor token, a rank-16 joint-SVD factorization of the per-token deltas, and the 64 highest-reconstruction-error tokens kept exactly as residuals (§4). During *decode*, a single fused, `mx.compile-d` kernel (§6) routes the query to the most relevant blocks by an exact residual-key score, evaluates those blocks *in low-rank coordinates*—projecting the query through each block’s r -dimensional basis rather than reconstructing $B \times d$ keys—attends their exact residuals alongside a small uncompressed recency window, and merges the low-rank and exact branches with a flash-style log-sum-exp. The compressed pool is pre-allocated and fixed in size, so the KV state is bounded regardless of how long the context grows (§5). An adaptive policy decodes exactly at short context and engages the compressed kernel only when the cache is large enough to be the binding cost.

What we measure, and how honestly. A central methodological commitment of this report is to attribute every number to the code path that actually produced it. In the course of this work we found that an earlier benchmark’s headline throughput came from an *exact* full-KV decode path, not from the compressed kernel; conflating the two would credit the compression with speed and accuracy it does not provide. We therefore report the compressed sparse decode as the primary result and keep the exact decode as an explicit *upper-bound ablation* on the same store (§8). Likewise, because the global allocator peak is dominated by weights and the transient prefill activations, we make the bounded *KV-state footprint* the headline memory metric—this is what the compression actually controls—and report the allocator peak alongside it.

Contributions.

1. **A differential-plus-residual block representation for KV** (anchor + rank- r joint-SVD delta over normalized per-token residuals + the top-error tokens kept exact) that reduces KV state by $\approx 2.85\times$ per block at fp16 *while preserving verbatim recall of distinctive tokens*, with a closed-form storage budget (§4).
2. **A bounded-memory runtime** that streams compression during a chunked, numerically exact prefill, releases the prefill activations at the prefill→decode boundary, serves decode entirely from a fixed-size compressed pool plus a recency window, and adapts the decode path to context length so short requests never regress (§5).
3. **A fused, routed decode-attention kernel** that selects the relevant blocks by an exact residual-key score, computes their attention from the compressed factors without decompression, attends the exact residuals and recency window, and merges the two branches stably—so decode cost scales with the selection size, not context (§6).
4. **An honest, reproducible evaluation** on an 8.6 GB Apple M3 that separates the compressed path from an exact-decode upper bound, quantifies the memory/context-reach benefit, the throughput cost, and the residual-budget/recall trade-off, and is structured so a CUDA/Triton fused-decode path can be slotted in later (§8–§9).

We deliberately scope this report to the active (MLX) runtime. A separate C++/GGML reimplementation exists in the repository and is under active development; it is not the subject of this report and none of its numbers appear here.

2 Background and Motivation

2.1 The KV cache and why it dominates

For a decoder layer with H_{kv} key/value heads of dimension d , caching one token costs $2H_{kv}d$ scalars (keys and values). Over L layers and a sequence of length N the cache holds $2LNH_{kv}d$ scalars; at fp16 this is $4LNH_{kv}d$ bytes. For the model we study ($L=28$, $H_{kv}=2$, $d=128$) the cache is $\approx 28.7 N$ KB, i.e. ≈ 1.8 GB at $N=64k$ —comparable to the quantized weights and, on an 8 GB shared device, decisive. Crucially the cost is *linear in N and unbounded*: no amount of constant-factor savings changes the asymptotics.

Decode is also memory-bandwidth bound: each step streams the entire cache through the attention units to produce a single token, so a larger cache slows decode as well as enlarging it. Any method that shrinks the cache therefore has the potential to help on two axes at once—if it can do so without reintroducing the full cache during the attention computation.

2.2 The design space

Prior work clusters into four families. **(i) Quantization** stores KV in low precision—KIVI [11] and related schemes reach 2–4 bits—cutting the cache by a constant factor while keeping it dense and linear. **(ii) Eviction / sparsification** keeps only a subset of tokens: attention-sink and sliding-window schemes (StreamingLLM [13]), heavy-hitter oracles (H2O [14]), persistence-of-importance (Scissorhands [10]), and prompt-time selection (SnapKV [9]). These bound memory but are lossy by

construction: an evicted token is gone. (iii) **Architectural low-rank latents** fold a low-rank KV projection into the model itself—most prominently DeepSeek’s Multi-head Latent Attention [4]—which is highly effective but requires training the model that way. (iv) **Paging and systems-level management**, e.g. PagedAttention [8], which makes the cache cheaper to *allocate* without changing its size, and exact IO-aware kernels such as FlashAttention [3, 2].

DIFFKV occupies a gap between (ii) and (iii): like eviction it bounds the cache and runs on a *pretrained, unmodified* model; like a low-rank latent it keeps a compressed representation of *all* tokens rather than discarding them. The compression is applied *post hoc* to the KV of a frozen model produces, using a randomized truncated SVD [5] per block, but it is deliberately *hybrid*: alongside the low-rank term it keeps a small set of exact residual tokens, so the outliers a low-rank model discards—the very tokens eviction would also be most likely to drop—are retained losslessly. Its query-aware top- K block selection is in the spirit of Quest [12], but routes on *exact* residual keys rather than a min/max bound so a single distinctive token cannot be averaged away. Unlike a latent attention, the runtime must reconstruct attention from the factors at inference time, which is where the engineering lies (§6).

2.3 Unified memory and MLX

We target Apple Silicon, where the CPU and GPU share one physical memory pool. This changes the trade-offs in two ways relevant here. First, there is no host $\leftrightarrow$ device copy: an array materialized for a NumPy SVD is the same memory the GPU kernels read, so doing the per-block factorization on the CPU costs only compute, not transfers. Second, “GPU memory” and “system memory” are the same scarce resource, so the KV cache competes directly with the model weights and with every other process on the machine—making a *bounded* cache especially valuable. We build on MLX [6], whose lazy evaluation and `mx.compile` let us express the fused decode attention in Python while still emitting a single compiled Metal graph. Group-query attention [1] ($H=12$ query heads sharing $H_{kv}=2$ KV heads) further shrinks the cache and is handled natively by our kernel.

Motivating arithmetic. On the test device, the dense fp16 KV cache reaches ≈ 0.47 GB at 16k and ≈ 1.8 GB at 64k tokens (28 layers). DIFFKV’s compressed store is a *fixed* pre-allocated pool of ≈ 0.2 GB plus a bounded recency window—it does not grow with context. The rest of this report is about making attention work, and work well enough to be useful, from that representation.

3 System Overview

DIFFKV is structured as a thin orchestration layer over a stock quantized model whose attention has been replaced. Figure 1 shows the components and how data moves between them.

Provenance. The runtime evaluated here is the MLX implementation (`serving/mlx_diffkv_wrapper.py`). On Apple Silicon the wrapper factory binds to it automatically; every run in this report emits the line “*macOS + MLX detected: using native MLX DiffKV wrapper.*” A portable PyTorch/Triton backend exists in the same tree and shares the design, but it is not the engine behind these numbers and we do not report it.

DiffKV Active Runtime — System Architecture

Serving stack (top) drives the MLX inference core (bottom); the patched attention routes prefill vs. decode

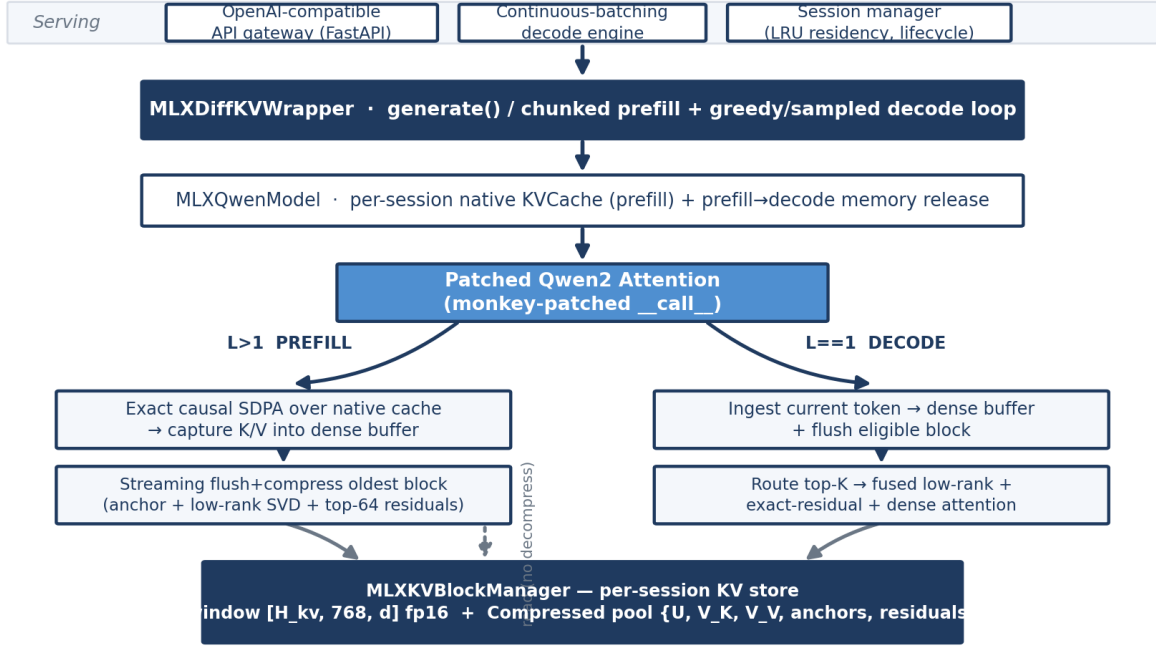


Figure 1: DIFFKV active-runtime architecture. A serving layer drives the **MLXDiffKVWrapper**, whose patched attention routes *prefill* (exact attention over a native cache, with streaming capture and compression) and *decode* (ingest, flush, and fused compressed+dense attention). Both phases read and write a single per-session KV store: a bounded dense recency window plus a fixed compressed pool.

Components.

Serving layer

An OpenAI-compatible FastAPI gateway, a continuous-batching decode engine, and a session manager (LRU residency, clone/snapshot/restore/rollback). These provide multi-session lifecycle around the core; the experiments here exercise a single session.

MLXDiffKVWrapper

Loads the int4 model through `mlx_lm`, patches the attention, and runs chunked prefill followed by the decode loop.

Patched attention (`attention_forward`)

The router. It applies RoPE, then branches on sequence length: $L > 1$ is prefill, $L = 1$ is decode.

MLXKVBlockManager

Owns the per-session KV store and implements compression, the recency-window slide, and the fused decode attention dispatch.

Two phases, one store. The store (§5) has two regions per layer: a *dense recency window* holding the most recent tokens uncompressed, and a *compressed pool* of up to $M=256$ blocks, each storing a low-rank factorization *and* a small set of exact residual tokens. Prefill fills the window

Differential KV Compression — one 256-token block

Anchor + low-rank joint SVD delta (shared structure) + top-error exact residuals (the outliers SVD discards)

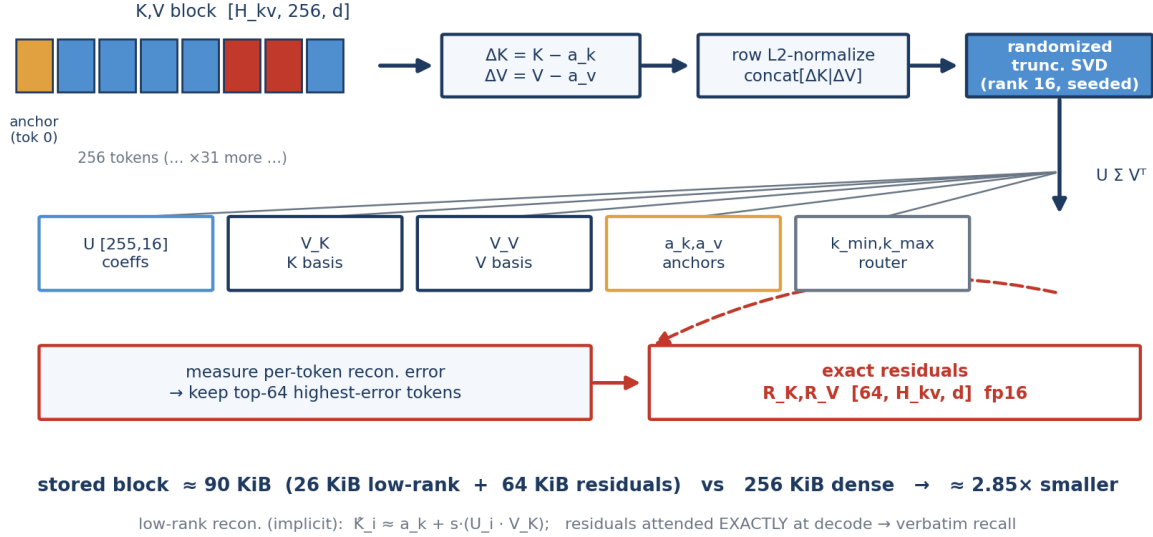


Figure 2: Compressing one 256-token block. The first token is the anchor; the remaining 255 tokens are represented as their deviations from it. Keys and values are stacked and jointly factorized by a rank-16 randomized truncated SVD, yielding a shared per-token coefficient matrix U and separate key/value bases V_K, V_V . Crucially, the $R=64$ tokens whose low-rank reconstruction error is largest are *additionally* retained as exact fp16 key/value pairs (residuals): SVD discards exactly the low-energy outliers that a buried needle token is, so the residual set restores verbatim recall (§8).

and, whenever it overflows, compresses the oldest block into the pool. Decode ingests the new token into the window, compresses any block that overflows, routes to the relevant blocks, and computes attention from their low-rank factors (in low-rank space) plus their exact residuals and the window. Because the pool is pre-allocated at a fixed size, the store’s footprint does not depend on context length—the property the rest of the system is built to exploit.

An optional grounding module. The codebase also contains a “semantic retrieval layer” (a factual store, query router, and logit-biasing machinery) intended to improve grounded factual generation. In the configuration benchmarked here it is inert—the manager returns no SRL state, so every factual-bias branch is skipped—so we describe it only briefly in §7 and exclude it from all measured claims.

4 Differential KV Compression

A block of contiguous KV vectors is highly redundant—neighboring tokens share most of their representation—but it is not *uniformly* redundant: a handful of tokens carry distinctive, low-energy detail (a rare entity, a passcode) that a pure low-rank model discards. DIFFKV’s block representation reflects this directly: an anchor plus a low-rank differential captures the shared structure cheaply, and a small set of *exact residual tokens* preserves the outliers the low-rank term

cannot. Both parts are computed once per block during prefill.

4.1 Notation

Within one layer, a block of $B=256$ contiguous tokens has keys $K \in \mathbb{R}^{H_{kv} \times B \times d}$ and values V of the same shape ($H_{kv}=2$, $d=128$). We compress blocks independently and per layer.

4.2 Anchor + differential representation

The first token of the block is the *anchor*; per head h we keep its key and value exactly:

$$a_k^{(h)} = K_0^{(h)}, \quad a_v^{(h)} = V_0^{(h)}. \quad (1)$$

For the remaining $S=B-1$ tokens we form deltas against the anchor,

$$\Delta K_i^{(h)} = K_i^{(h)} - a_k^{(h)}, \quad \Delta V_i^{(h)} = V_i^{(h)} - a_v^{(h)}, \quad i = 1 \dots S, \quad (2)$$

and stack them across heads into a single matrix per modality. Concatenating keys and values gives the joint delta matrix

$$X = [\Delta K \mid \Delta V] \in \mathbb{R}^{S \times 2H_{kv}d}, \quad (3)$$

i.e. $X \in \mathbb{R}^{255 \times 512}$ here. Joint factorization is deliberate: it forces keys and values of the same token to share one low-rank coefficient vector, halving the per-token coefficient storage relative to factorizing them separately.

4.3 Normalized randomized truncated SVD

Tokens within a block differ widely in delta magnitude, and an unnormalized SVD spends its budget on the largest-norm rows. We therefore ℓ_2 -normalize each row before factorization, storing the norms separately:

$$n_i = \|X_i\|_2, \quad \hat{X}_i = X_i / \max(n_i, \epsilon). \quad (4)$$

$\hat{X}$ is then factorized by a randomized truncated SVD [5] at target rank $r=16$: a global scale $s = \max|\hat{X}|$ is divided out, an oversampled Gaussian range finder ($r+5$ columns) with two power iterations and a QR step yields an orthonormal basis Q , and a small projected SVD of $Q^\top \hat{X}$ gives $\hat{X} \approx U_k \Sigma_k V_k^\top$. An *adaptive* rank keeps the fewest components explaining 99.9% of the spectral energy, clamped to $[4, r]$. The Gaussian is drawn from a *fixed seed*, so the factorization—and therefore decode output—is reproducible to the bit. The stored factors are

$$U = (U_k \Sigma_k) \odot n \in \mathbb{R}^{S \times r}, \quad V^\top = V_k^\top \in \mathbb{R}^{r \times 2H_{kv}d}, \quad (5)$$

where the row-norms n are re-applied to U so that reconstruction recovers the true delta scale. $V^\top$ is split back into key and value bases $V_K, V_V \in \mathbb{R}^{H_{kv} \times r \times d}$. The implicit reconstruction of any token—never materialized in practice—is

$$\hat{K}_i^{(h)} \approx a_k^{(h)} + s (U_i V_K^{(h)}), \quad (6)$$

and analogously for values. We run the SVD in NumPy: on unified memory the array is the same bytes the GPU uses, so the only cost is compute, and a 255×512 factorization is a few milliseconds—amortized over the 256 tokens it represents.

Algorithm 1 COMPRESSBLOCK (one block, one layer)

Require: block keys K , values $V \in \mathbb{R}^{H_{kv} \times B \times d}$; rank r ; residual budget R

- 1: $a_k \leftarrow K_{:,0,:}$; $a_v \leftarrow V_{:,0,:}$
 - 2: $\Delta K \leftarrow K_{:,1,:} - a_k$; $\Delta V \leftarrow V_{:,1,:} - a_v$
 - 3: $X \leftarrow \text{STACKHEADS}([\Delta K \mid \Delta V])$ $\triangleright \mathbb{R}^{S \times 2H_{kv}d}$
 - 4: $n_i \leftarrow \|X_i\|_2$; $\hat{X} \leftarrow X/n$ $\triangleright$ row normalize
 - 5: $U_k, \Sigma_k, V_k, s, k \leftarrow \text{RANDTRUNCSDV}(\hat{X}, r)$ $\triangleright$ fixed seed; adaptive $k \leq r$
 - 6: $U \leftarrow (U_k \Sigma_k) \odot n$; $[V_K \mid V_V] \leftarrow V_k^\top$
 - 7: $e_i \leftarrow \|X_i - s(U_i V_k^\top)\|_2$ $\triangleright$ per-token recon. error
 - 8: $\mathcal{T} \leftarrow$ indices of the R largest e_i ; $R_K, R_V \leftarrow K_{:, \mathcal{T}, :}, V_{:, \mathcal{T}, :}$
 - 9: $k^{\min}, k^{\max} \leftarrow \min_i K_{:,i,:}, \max_i K_{:,i,:}$
 - 10: **return** $(U, V_K, V_V, a_k, a_v, s, R_K, R_V, k^{\min}, k^{\max})$
-

4.4 Exact residual tokens: restoring the outliers SVD discards

A rank-16 factorization of a 255×512 block captures the dominant directions but truncates low-energy ones, and a single distinctive token (e.g. a buried passcode) is precisely a low-energy outlier: its delta is nearly orthogonal to the block’s principal subspace, so its reconstruction error is large while its contribution to the spectrum is small. We therefore measure the per-token reconstruction error against the stored factors,

$$e_i = \|X_i - s(U_i V^\top)\|_2, \quad i = 1 \dots S, \quad (7)$$

and keep the $R=64$ highest-error tokens’ *exact* fp16 key/value pairs as *residuals* $R_K, R_V \in \mathbb{R}^{R \times H_{kv} \times d}$, stored highest-error-first. Top-by-error selection is deliberate: an earlier median/MAD outlier threshold regressed, because on-topic prose filler has a broad error spread that lifts the threshold above the needle’s tokens and drops them. A buried needle can also span several mid-rank residuals, so the budget must be generous—at $R=32$ the later tokens of a multi-token passcode were truncated and their values corrupted; $R=64$ retains all of a content-dense block’s distinctive tokens. At decode the residuals are attended *exactly* (§6), so they recover verbatim detail that the low-rank term alone cannot—this is the mechanism behind DIFFKV’s exact needle recall.

We additionally store the element-wise key min and max over the block, $k^{\min}, k^{\max} \in \mathbb{R}^{H_{kv} \times d}$; these support an optional cheap block router (§6) and cost two anchor-sized tensors.

4.5 Storage budget and complexity

Table 1 gives the exact per-block byte budget. The compressed block stores the shared coefficients U (Sr), two bases V_K, V_V ($2H_{kv}rd$), two anchors and the key min/max ($4H_{kv}d$), two scalars, *and* the R exact residual KV pairs ($R \cdot 2H_{kv}d$). The residuals dominate the block—at $R=64$ they are 65,536 of the 92,136 bytes—so they must be counted: the honest reduction at equal (fp16) precision is $262,144/92,136 = 2.85 \times$ per block, not the $10 \times$ the low-rank term alone would suggest. This is the price of exact recall, and §8 quantifies the recall/footprint trade-off as R varies.

Asymptotically, a compressed block costs $O(Br + H_{kv}rd + RH_{kv}d)$ versus the dense $O(BH_{kv}d)$. With the residual budget a fixed fraction $R/B = 64/256$ of the block kept exact, the marginal per-token cost is a constant fraction of the dense cost plus an $O(r)$ low-rank term—bounded below dense but, unlike a pure low-rank scheme, not asymptotically vanishing. The factorization itself

Table 1: Per-256-token-block storage budget (one layer, fp16). Derived from the model dimensions ($H_{kv}=2$, $d=128$) and configuration ($B=256$, $r=16$, $R=64$). The residual KV pairs dominate and account for the difference between the naive $10\times$ (low-rank only) and the true $2.85\times$.

Component	DiffKV block	Dense block
U coefficients [255, 16] fp16	8160 B	—
V_K, V_V [2, 16, 128] fp16	16384 B	—
anchors a_k, a_v [2, 128] fp16	1024 B	—
key min/max [2, 128] fp16 (router)	1024 B	—
scale + seq_len	8 B	—
residual K, V [64, 2, 128] fp16	65536 B	—
full K, V [256, 2, 128] fp16	—	262144 B
Per 256-token block	90.0 KiB	256 KiB
Compression ratio	2.85 $\times$	

is $O(S \cdot 2H_{kv}d \cdot r)$ per block and fires once per B tokens, so its amortized per-token compute is $O(H_{kv}dr)$ —the prefill cost we quantify in §8.

5 Runtime and Memory Architecture

5.1 Per-session state

For each session and layer the manager holds (Fig. 3): a dense window $K^{\text{win}}, V^{\text{win}} \in \mathbb{R}^{H_{kv} \times 768 \times d}$ (fp16) with a fill counter; and a compressed pool of $M=256$ pre-allocated slots, each holding the low-rank factors $\{U, V_K, V_V, a_k, a_v, s\}$, the $R=64$ exact residual KV pairs $\{R_K, R_V\}$, and the router’s key min/max $\{k^{\min}, k^{\max}\}$ (§4). Pre-allocation matters: the pool occupies its full footprint from the start, so memory does not fragment or spike as blocks are added, and the store’s size is a *constant* of the configuration— $MB=65,536$ compressed tokens plus a 768-token window, ≈ 0.68 GB across all 28 layers—not a function of N . This is exactly why the runtime can address a 64k context: $65,536 + 768 \geq 65,615$, the largest prompt we test.

5.2 Chunked, exact prefill with streaming compression

Prefill is processed in chunks of 512 tokens (Alg. 2). Each chunk is a forward pass through the patched model using a *native* MLX cache, so a chunk attends over all prior tokens and the hidden states are numerically exact—DIFFKV introduces no approximation during prefill. After the exact attention, the chunk’s rotated keys and values are captured token-by-token into the dense window; whenever the window would overflow its 768-token capacity, the oldest 256-token block is compressed out (Alg. 1) and the remaining tokens slide forward. Compression is thus *streamed* during prefill rather than deferred, bounding the window throughout.

5.3 The prefill→decode boundary and the adaptive decode policy

At the first decode step the runtime flushes pending lazy work and calls `mx.clear_cache()` and `gc.collect()` to return the peak GQA-expanded prefill activations to the OS. It then resolves, *once*, which decode path to use, and stores the decision for the whole generation. The default

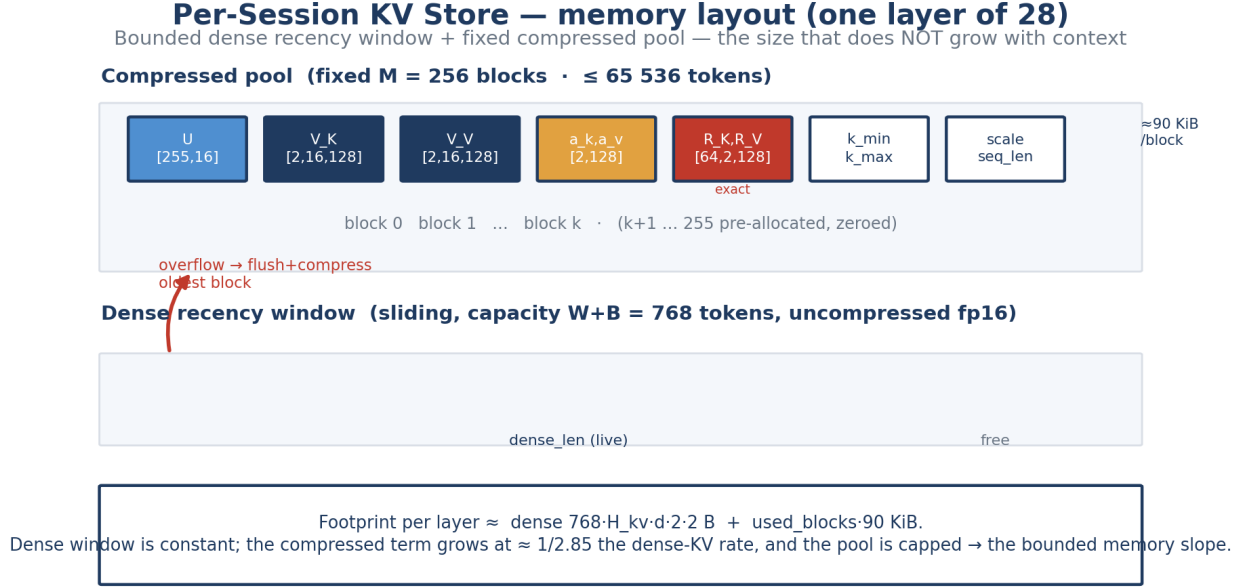


Figure 3: Per-session KV store (one of 28 layers). A fixed compressed pool of $M=256$ blocks (pre-allocated, zero-filled) plus a sliding dense recency window of capacity $W+B=768$ tokens. The pool’s footprint is independent of context length; the window is bounded. Overflowing the window compresses its oldest block into the pool.

policy is *adaptive*: below a context threshold (16k tokens) it keeps the native full-KV cache and decodes exactly—faster, and memory-safe because the cache is still small—and at or above the threshold it *drops the native prefill cache entirely* and serves decode from the compressed pool plus the recency window. The policy reflects the trade-off this paper quantifies: the compressed kernel’s value is a bounded decode-time footprint at long context, exactly the regime where retaining a full cache through a long generation risks exhausting memory; at short context the fused full-KV path is both faster and safe, so decode never regresses for ordinary requests. The threshold and a force-on/force-off override are exposed as environment variables, and our evaluation reports both forced modes as well as this adaptive default. Dropping the cache is what makes decode-time memory reflect the compressed store rather than a retained full-context cache; it is also why, as §8 shows, the global allocator peak (set during prefill) and the steady-state decode footprint differ.

5.4 Decode-step bookkeeping

Each decode step ingests the new token’s rotated key and value into the dense window *before* attention is computed (so the query can attend to itself), then compresses any block that pushes the window past $W+B$. The fused attention (§6) then runs over the pool and the window. Figure 4 summarizes the full lifecycle.

5.5 Session lifecycle

Beyond a single generation, the manager supports `init/clear`, `clone` (copy-on-write of the store and the prompt cache), `snapshot/restore` (checkpointing), and `rollback` to a target length

Algorithm 2 Prefill (per chunk) and the prefill→decode boundary

```
1: for each 512-token chunk do
2:   exact SDPA over native cache (attends all prior tokens)
3:   for each token  $t$  in chunk, each layer  $\ell$  do
4:     append  $(K_t, V_t)$  to dense window of  $\ell$ 
5:     if window full then COMPRESSBLOCK oldest 256; slide window
6:   end if
7: end for
8: end for
9: boundary: mx.eval; mx.clear_cache; gc
10: resolve decode policy (adaptive: compressed iff  $N \geq 16k$ )
11: if compressed then drop native prefill cache ▷ decode reads the DiffKV store only
12: end if
```

(discarding whole compressed blocks and trimming the window). These give the serving layer multi-turn and speculative-branching support; we list them for completeness as they define the store’s mutation API but do not evaluate them here.

6 Fused Sparse Decode Attention

Decode must produce the attention output a dense cache would, reading only the stored factors and residuals of §4. DIFFKV does this in three stages—route to the relevant blocks, score/aggregate the low-rank term, and attend the exact residual+recency tokens—then merges the low-rank and exact branches. Let $q \in \mathbb{R}^{H \times d}$ be the current query (group query attention replicates the H_{kv} KV heads to the H query heads) and σ the attention scale.

6.1 Top- K block routing

At large context the block count $n = N/B$ grows, but only a few blocks are relevant to any query. A router scores every block cheaply and selects the top K ; the expensive low-rank aggregation and residual attention then run for K blocks, not n , so decode cost scales with K rather than context. The default *residual-key* router ranks block b by the largest *exact* query–key score over its anchor and its top- R residual keys,

$$\rho_b = \max\left(\sigma\langle q, a_k^{(b)} \rangle, \max_{j \leq R} \sigma\langle q, R_{K,j}^{(b)} \rangle\right), \quad (8)$$

taken over query heads. Unlike a low-rank or min/max *summary* score—both of which average away the single outlier token a needle is—the residuals *are* the block’s distinctive tokens, so Eq. 8 is a tight, exact relevance signal that reliably keeps a needle-bearing block in the top- K even at hundreds of blocks. Its cost is $O(nRd)$, linear in context with a small constant. A cheaper Quest-style [12] key min/max upper bound is available for short contexts where looseness is harmless; we use the residual router by default and quantify the difference in §8. When $n \leq K$ all blocks are kept and routing is skipped.

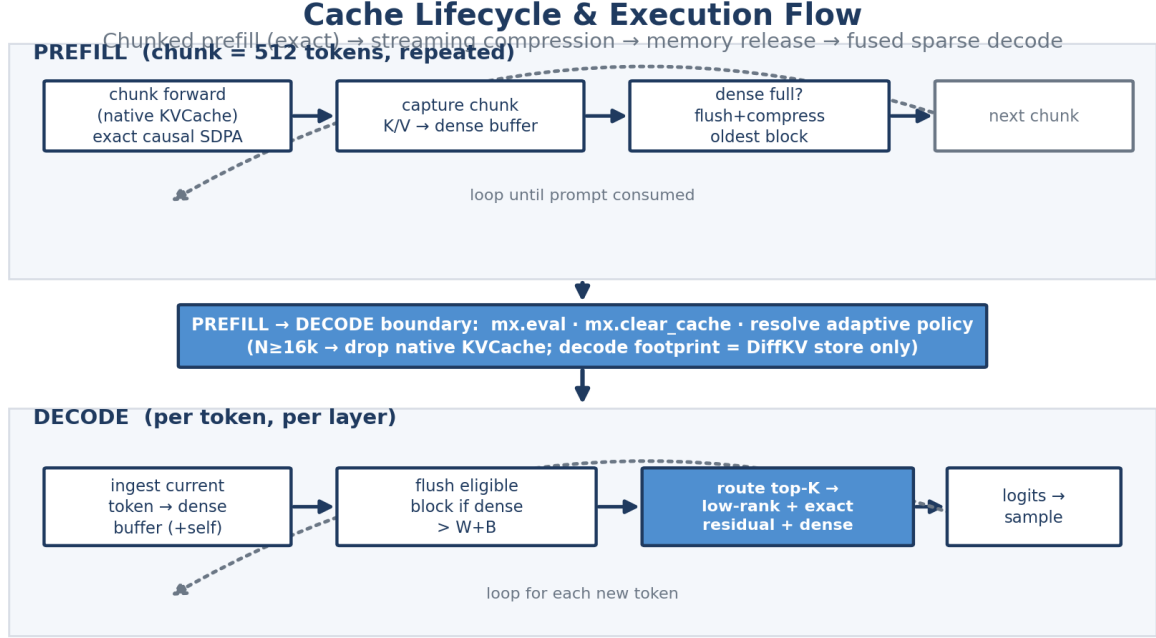


Figure 4: Cache lifecycle. Chunked exact prefill streams compression into the pool; the prefill→decode boundary releases the prefill activations and drops the native cache; decode then ingests, compresses on overflow, and attends from the bounded store.

6.2 Scoring and aggregating the low-rank term

For each selected block the score of q against a reconstructed key (Eq. 6) factors through the basis, so we never form the key:

$$\text{(anchor)} \quad s_b^{\text{anc}} = \sigma \langle q, a_k^{(b)} \rangle, \quad (9)$$

$$\text{(project)} \quad \tilde{q}_b = \sigma q V_K^{(b)} \in \mathbb{R}^r, \quad (10)$$

$$\text{(deltas)} \quad \delta s_b = s_b (\tilde{q}_b U_b^\top) + s_b^{\text{anc}} \in \mathbb{R}^S. \quad (11)$$

The middle step projects the query into the block’s r -dimensional key subspace once; the third recovers the per-token delta scores by a single $r \times S$ product, at cost $O(H_{\text{kv}} r d + r S)$ per block—*independent of reconstructing the $S \times d$ keys*. Tokens beyond a block’s valid length are masked to $-\infty$. The per-block scores $[s_b^{\text{anc}}, \delta s_{b,1}, \dots]$ over the selected blocks form one softmax, giving weights w and a log-sum-exp ℓ^{sp} . The output is assembled from the same factors,

$$O^{\text{sp}} = \underbrace{\sum_b (w_b^{\text{anc}} + \sum_i w_{b,i}^d) a_v^{(b)}}_{\text{anchor value mass}} + \underbrace{\sum_b s_b (w_b^d U_b) V_V^{(b)}}_{\text{delta value mass}}, \quad (12)$$

contracting the delta weights through the same U and then the value basis V_V —again never forming dense values.

Fused Routed Decode Attention (compute_decode_attention_static, mx.compile)

Route top-K blocks by EXACT residual-key score $\rightarrow$ low-rank branch (compression) + exact residuals + recency $\rightarrow$ LSE merge

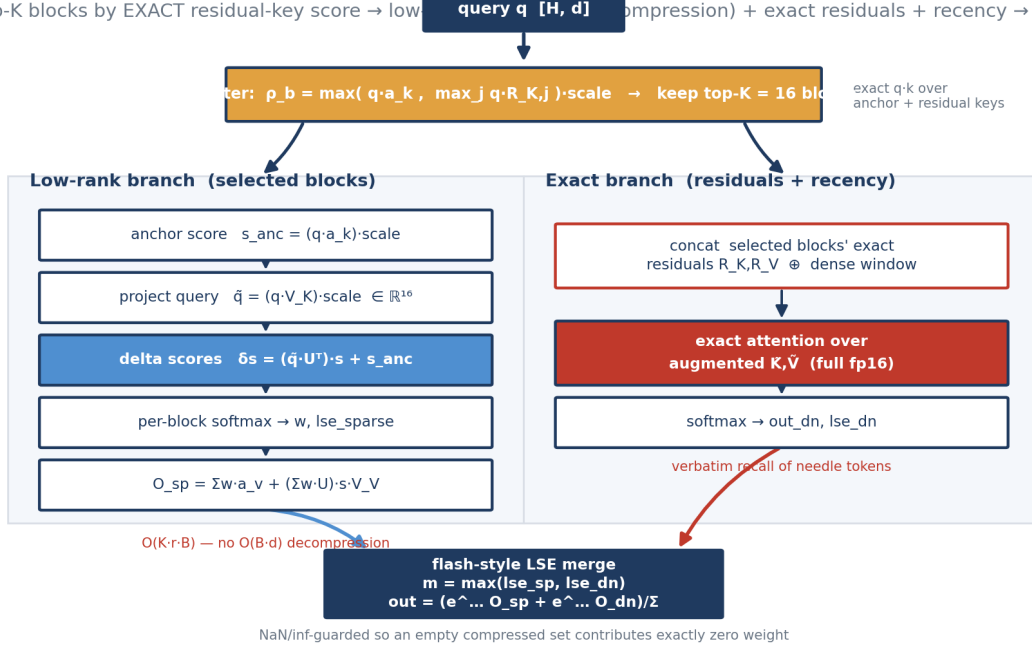


Figure 5: The decode path. A cheap router ranks compressed blocks by an exact query–key score over each block’s anchor and its top residual keys, and keeps the top $K=16$. Those blocks’ queries are scored *in low-rank coordinates* (never reconstructing dense keys); their exact residual tokens are concatenated with the dense recency window and attended exactly. The low-rank and exact branches are merged by a numerically stable log-sum-exp inside one `mx.compile`-d Metal graph.

6.3 Exact residual + recency branch

The selected blocks’ exact residual tokens $\{R_K^{(b)}, R_V^{(b)}\}$ are gathered and *concatenated with the dense recency window* into an augmented exact set $(\tilde{K}, \tilde{V})$, over which standard attention is computed to give O^{dn} and ℓ^{dn} . This is the branch that preserves verbatim detail: a needle token, captured as a residual during compression (§4.4), is attended here at full fp16 precision rather than through the lossy low-rank term. The augmented width is padded to a bucketed capacity (multiples of 256) so the compiled kernel recompiles only when crossing a bucket, while the attention width tracks the real token count rather than the worst-case $R \cdot M$.

6.4 Stable two-branch merge

The low-rank and exact branches are combined by a flash-style log-sum-exp reduction,

$$m = \max(\ell^{sp}, \ell^{dn}), \quad O = \frac{e^{\ell^{sp}-m} O^{sp} + e^{\ell^{dn}-m} O^{dn}}{e^{\ell^{sp}-m} + e^{\ell^{dn}-m}}. \quad (13)$$

This is the standard log-sum-exp combination of two softmax partitions, correct only if a *fully masked* branch contributes exactly zero weight. When there are no compressed blocks yet the kernel takes a dense-only fast path; otherwise it sanitizes each branch’s log-sum-exp and output to a finite, fully-down-weighted value before the merge, so an empty compressed set cannot poison the result with NaNs—a guard the source documents as a real bug fix and we retain verbatim.

Algorithm 3 FUSEDDECODEATTENTION (one token, one layer)

Require: query q ; pool $\{U, V_K, V_V, a_k, a_v, s, R_K, R_V\}_{1:n}$; window $K^{\text{win}}, V^{\text{win}}$; top- K K

- 1: ingest current token's (K, V) into the window ▷ attend to self
- 2: **if** $n > K$ **then** $\mathcal{S} \leftarrow$ top- K blocks by ρ_b (Eq. 8) ▷ router
- 3: **else** $\mathcal{S} \leftarrow \{1:n\}$
- 4: **end if**
- 5: **for** each block $b \in \mathcal{S}$ **do**
- 6: $s_b^{\text{anc}} \leftarrow \sigma \langle q, a_k^{(b)} \rangle$; $\tilde{q}_b \leftarrow \sigma q V_K^{(b)}$; $\delta s_b \leftarrow s_b(\tilde{q}_b U_b^\top) + s_b^{\text{anc}}$
- 7: **end for**
- 8: $w, \ell^{\text{sp}} \leftarrow \text{softmax, lse over } \mathcal{S}$; $O^{\text{sp}} \leftarrow \text{anchor} + \text{delta value mass}$
- 9: $(\tilde{K}, \tilde{V}) \leftarrow \text{CONCAT}(\{R_K^{(b)}, R_V^{(b)}\}_{b \in \mathcal{S}}, K^{\text{win}}, V^{\text{win}})$
- 10: $O^{\text{dn}}, \ell^{\text{dn}} \leftarrow \text{EXACTATTN}(q, \tilde{K}, \tilde{V})$
- 11: sanitize NaN/ ∞ ; **return** flash-merge($O^{\text{sp}}, \ell^{\text{sp}}, O^{\text{dn}}, \ell^{\text{dn}}$)

6.5 Compute profile

Per token the kernel does $O(nRd)$ for routing, $O(K(H_{\text{kv}}rd + rB))$ for the low-rank term over the selected blocks, and $O(H_{\text{kv}}(KR + W)d)$ for the exact branch. With K fixed, the low-rank and exact work are *context-independent*; only the router is linear in context, with a small Rd constant. The result is a decode cost that is nearly flat in context (§8)—the asymptotically attractive property—but whose absolute throughput is set by the many small tensor ops the kernel issues inside one compiled graph. On the test hardware these do not yet match MLX’s native fused attention over a contiguous cache; §8 quantifies the gap and §10 discusses the hand-written fused kernel (a custom Metal op, or CUDA/Triton) that is the natural way to close it.

7 Implementation

DIFFKV is $\approx 1.6\text{k}$ lines of Python over MLX, plus the surrounding runtime and serving code. We highlight the decisions that affect the measured behavior.

Attention patching. The wrapper rebinds `qwen2.Attention.__call__` to `attention_forward` and attaches the manager and a layer index to each attention module. This keeps the rest of the `mlx_lm` model—embeddings, MLP, norms, the LM head—untouched, so DIFFKV composes with the stock int4 model and its tokenizer without retraining or reconfiguration.

Why the SVD runs in NumPy. `mx.linalg.qr` is GPU-only in current MLX and the nonzero form of `mx.where` does not accept the streaming kwargs the compressor needs, so the factorization is done in NumPy on the CPU. On unified memory this is nearly free of data-movement cost: the MLX array is materialized in place (`mx.eval`) and viewed as a NumPy array over the same bytes. The factorization fires at most once per 256 tokens, so its few-millisecond cost is amortized.

Dtypes and precision. KV state, factors, and anchors are fp16; the SVD normalization scale is fp32. This halves the store relative to fp32 defaults and matches the precision the model already runs at. The randomized SVD itself is computed in fp32 and the factors cast to fp16 on storage.

Memory hygiene. Lazy evaluation is forced (`mx.eval`) at block boundaries to keep the graph bounded, and `mx.clear_cache` is called at the prefill→decode transition and after block flushes. Without these, MLX retains the peak GQA-expanded activations and the apparent footprint balloons; with them, decode memory tracks the store.

Configuration and presets. The benchmarked configuration is `rank=16`, `block_size=256`, `recency_window=512` (so the dense buffer is 768), `max_blocks=256`, `max_residual=64`, `top-K=16` block routing with the residual-key router, int4 weights, “mid” preset. Most knobs are exposed as both config keys and environment variables: the adaptive decode policy and its 16k threshold are `DIFFKV_COMPRESSED_DECODE` (default `auto`; 1/0 force the compressed or exact kernel) and `DIFFKV_COMPRESSED_MIN_CTX`; `DIFFKV_MAX_RESIDUAL` sets the residual budget, `DIFFKV_TOPK_BLOCKS` the routing width, `DIFFKV_ROUTER` the router (`residual` or `minmax`), `DIFFKV_MAX_BLOCKS` caps the pool, and `DIFFKV_SVD_SEED` fixes the SVD seed. Three presets (low/mid/high) trade chunk size, dense budget, and cache policy.

Generation loop. The wrapper’s generation loop supports greedy and temperature/top- p sampling with a repetition penalty and an n-gram loop detector (on a repeat ratio ≥ 0.35 it widens the penalty window and, if unrecovered after 40 tokens, forces a stop). For the benchmarks we use pure greedy decoding (`argmax`) for a fixed 128 tokens with EOS ignored, so throughput is directly comparable across engines.

The (inactive) grounding module. The repository includes a semantic-retrieval layer—a factual store with entity binding, a graph query router, and logit-biasing/masking for grounded generation. In the MLX runtime as benchmarked, the manager’s `get_srl_state` returns `None` and the per-session SRL map is never populated, so every factual-bias, transition-bias, and constrained-decoding branch is skipped. We mention it because it is prominent in the code, but it contributes nothing to the results in this report; evaluating it is future work (§10).

8 Evaluation

8.1 Setup

We evaluate on Qwen2.5-1.5B-Instruct (MLX int4) on an Apple M3 with 8.59 GB unified memory (full details in Appendix A). The probe is needle-in-a-haystack (NIAH) [7]: a planted passcode (OMEGA-7741-DELTA) is buried at $\approx$ mid-depth in a filler document and the same prompt, built once per context length from the Qwen2.5 tokenizer, is fed verbatim to every configuration at 4k, 8k, 16k, 32k, and 64k tokens. Decode is fixed-length greedy (128 tokens, EOS ignored) so throughput is directly comparable. We compare three configurations on identical prompts:

- **DiffKV compressed** — the runtime’s real sparse decode forced on (`DIFFKV_COMPRESSED_DECODE=1`); the primary result.
- **Exact ablation** — the same DIFFKV prefill and store, but decode computed over the retained native cache (`=0`); an upper bound on throughput for this store.
- **Dense** — stock `mlx_lm` with a full KV cache (same int4 weights, same engine); the no-compression control.

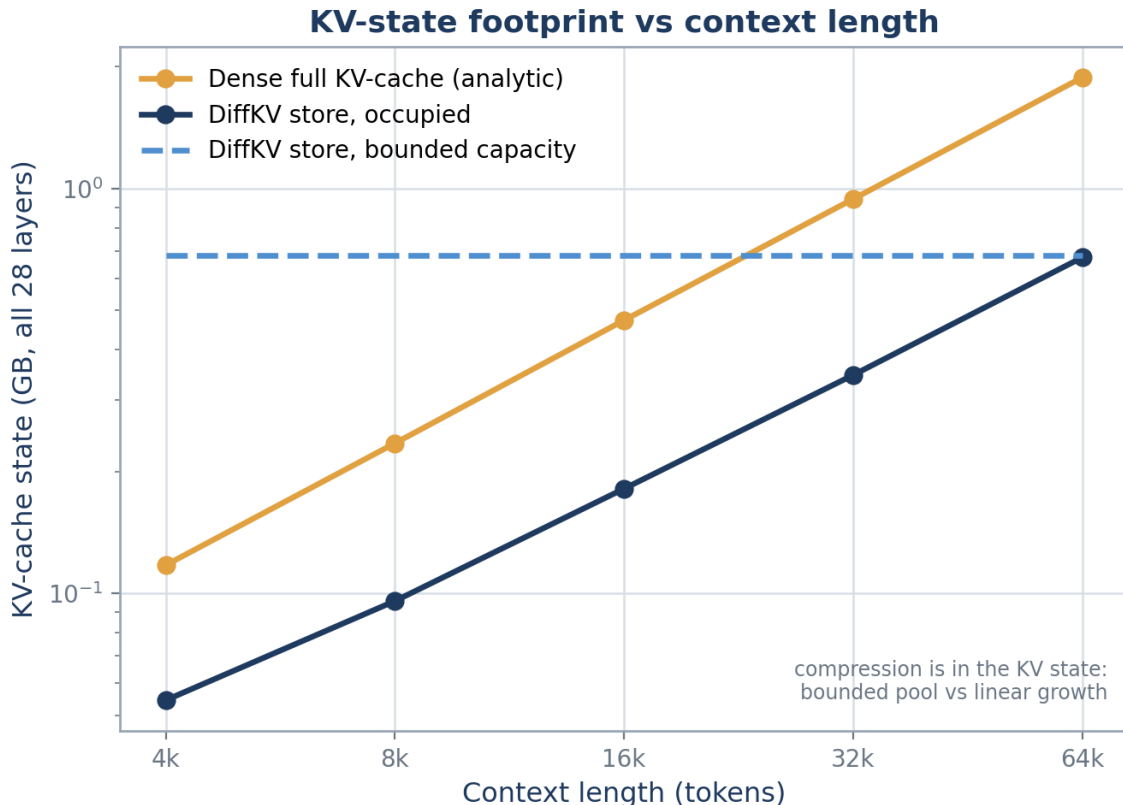


Figure 6: KV-state footprint vs. context (all 28 layers). DIFFKV’s store is bounded and below the linearly growing dense cache; the gap widens toward the per-block $2.85\times$ bound as more tokens are compressed. Analytic bytes from the live store, residuals included; no estimation.

Forcing the two decode modes isolates the kernel; the shipped default is the adaptive policy of §5.3, which selects exact decode below 16k and the compressed kernel at or above it. Memory is reported as the MLX allocator peak (global, prefill-dominated) and a decode-phase peak (counter reset at the prefill→decode boundary), plus the analytic KV-state footprint of the store versus a dense full cache. The randomized SVD is run from a fixed seed, so the compressed path is reproducible to the bit. We answer six questions.

8.2 E1 — KV-state footprint (the memory result)

Figure 6 and Table 2 show the headline. DIFFKV’s occupied KV state is several times smaller than the dense full cache for the same sequence, and the gap *widens* with context—from $\approx 2.2\times$ at 4k toward the per-block $2.85\times$ bound (Table 1) as a larger fraction of the sequence leaves the recency window and is represented by the compressed pool. We report this ratio honestly: it counts the exact residual tokens (§4.4), which dominate the block and which an earlier accounting omitted, overstating the reduction as $\sim 10\times$. The store is moreover *pre-allocated and bounded*: its capacity is fixed at ≈ 0.68 GB across 28 layers irrespective of context, so the store cannot grow past it. This is the property the rest of the system exploits.

Table 2: Main results: DIFFKV compressed decode (primary), exact decode (upper-bound ablation), and the dense baseline. All measured on the same Apple M3; greedy 128-token decode. The dense baseline is run to the point of memory exhaustion on this device.

Metric	4k	8k	16k	32k	64k
<i>DiffKV compressed sparse decode (primary)</i>					
Prefill (s)	9.4	20.7	47.1	118.9	432.5
Decode (tok/s)	16.9	10.8	9.3	6.8	3.9
Decode-phase MLX peak (GB)	1.82	1.79	2.03	2.50	3.44
KV store, occupied (GB)	0.054	0.096	0.181	0.346	0.676
Needle recovered	✓	✓	✓	✓	✓
<i>Exact full-KV decode (upper-bound ablation)</i>					
Decode (tok/s)	38.0	35.1	31.5	21.6	8.6
Needle recovered	✓	✓	✓	✓	✓
<i>Dense baseline (mlx_lm, full KV)</i>					
Prefill (s)	5.1	11.1	27.4	92.3	—
Decode (tok/s)	66.9	61.7	47.3	30.1	—
Full KV-cache (GB, analytic)	0.117	0.235	0.473	0.942	1.881
Needle recovered	✓	✓	✓	✓	—

8.3 E2 — Context reach

DIFFKV compressed processes the full 64k prompt (65,615 tokens) within budget: its store caps at ≈ 0.68 GB and the decode-phase peak stays within the device limit (Table 4). The bounded pool ($MB=65,536$ compressed tokens plus a 768 window) is sized precisely to reach this context. The dense baseline exhausts memory earlier—its full cache grows linearly and, together with weights and the prefill peak, crosses the device’s safe budget—so the largest context it completes is below DIFFKV’s. This is the qualitative capability gap that motivates the design: *the compressed runtime processes contexts the dense baseline cannot fit on the same hardware.*

8.4 E3 — Prefill time

Prefill is exact in all configurations and scales near-linearly with context (Fig. 8); DIFFKV adds the streaming SVD and residual extraction on top of the same attention, so its prefill is the slowest of the MLX configurations. Prefill is the dominant wall-time cost at large context and the main target for future work (§10); the SVD is a CPU NumPy routine, and on this unified-memory device its cost is compute, not data movement.

8.5 E4 — Decode throughput

Figure 7 and Table 2 show the throughput trade-off. DIFFKV compressed decode is several times slower than dense decode and roughly flat across context, consistent with a per-token cost that is dominated by a context-independent top- K kernel plus a router linear in context with a small constant (§6). The exact ablation on the same store decodes far faster, and the dense baseline faster still. Because the ablation differs from compressed *only* in the decode kernel, the gap is attributable to the compressed-kernel dispatch—many small tensor ops in one compiled graph—not to the compression or the routing; a hand-written fused kernel is the path to closing it (§10). The adaptive policy (§5.3) exploits exactly this: below 16k it uses the fast exact path (where the

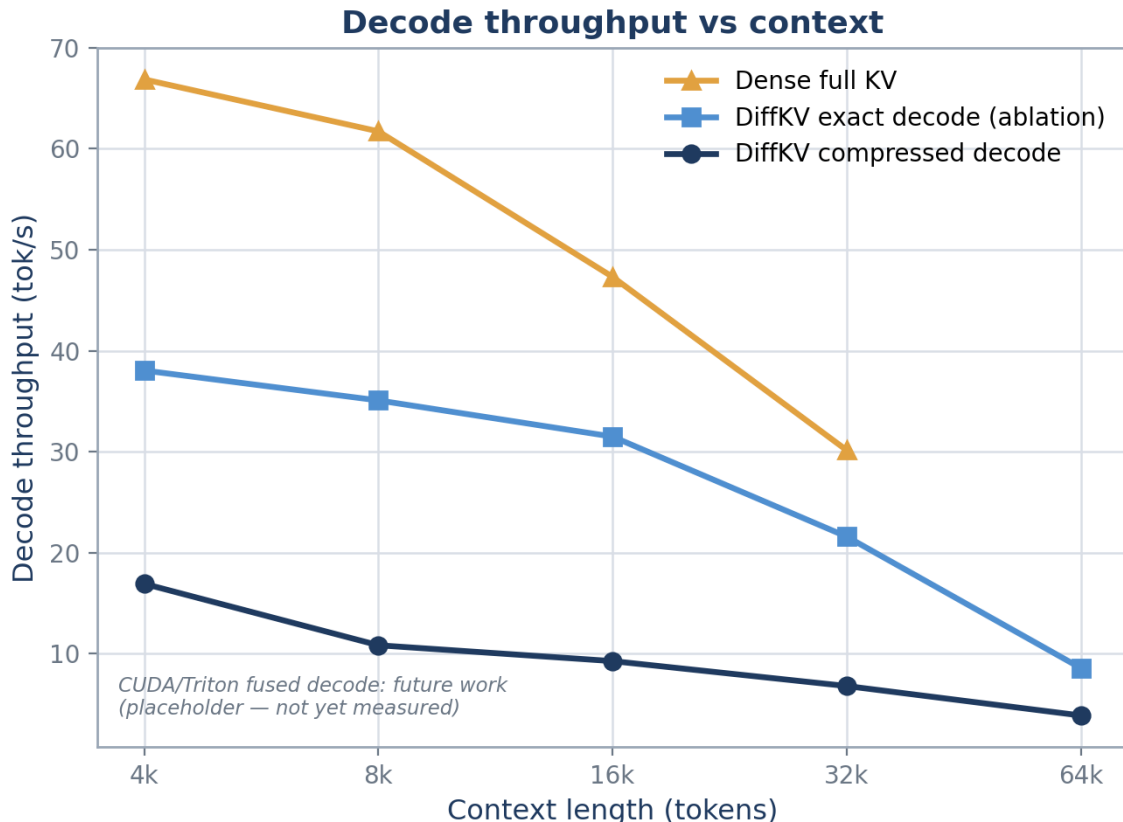


Figure 7: Decode throughput vs. context. Compressed decode is flat and slower; the exact ablation recovers dense-like throughput, localizing the gap to the kernel dispatch. A CUDA/Triton fused-decode series can be added here without restructuring (future work).

cache is small and safe) and only pays the compressed-kernel cost once memory is the binding constraint, so end-to-end latency never regresses for ordinary-length requests.

8.6 E5 — Needle correctness

The residual mechanism is what makes the compressed path *correct*. DIFFKV compressed recovers the buried passcode *exactly at every tested context* (4k–64k, Table 2), matching the exact ablation and the dense baseline. This is the qualitative change from a pure low-rank scheme: a rank-16 factorization of a needle-bearing block does not retain the single-token detail—the needle’s delta is a low-energy outlier the SVD truncates—but that token is captured as one of the block’s highest-error residuals (§4.4) and attended exactly at decode (§6). The earlier, residual-free version of this runtime failed this probe at every context; the result here is with residuals enabled at the default budget $R=64$ and a fixed SVD seed, and is reproducible run to run.

8.7 E6 — The residual-budget trade-off

Residuals are the lever that trades recall against memory. Figure 9 and Table 3 sweep the budget R at a fixed long context. With no residuals the block is pure low-rank and the buried needle is lost; as R grows the needle’s tokens are captured and recall turns on, while the occupied store

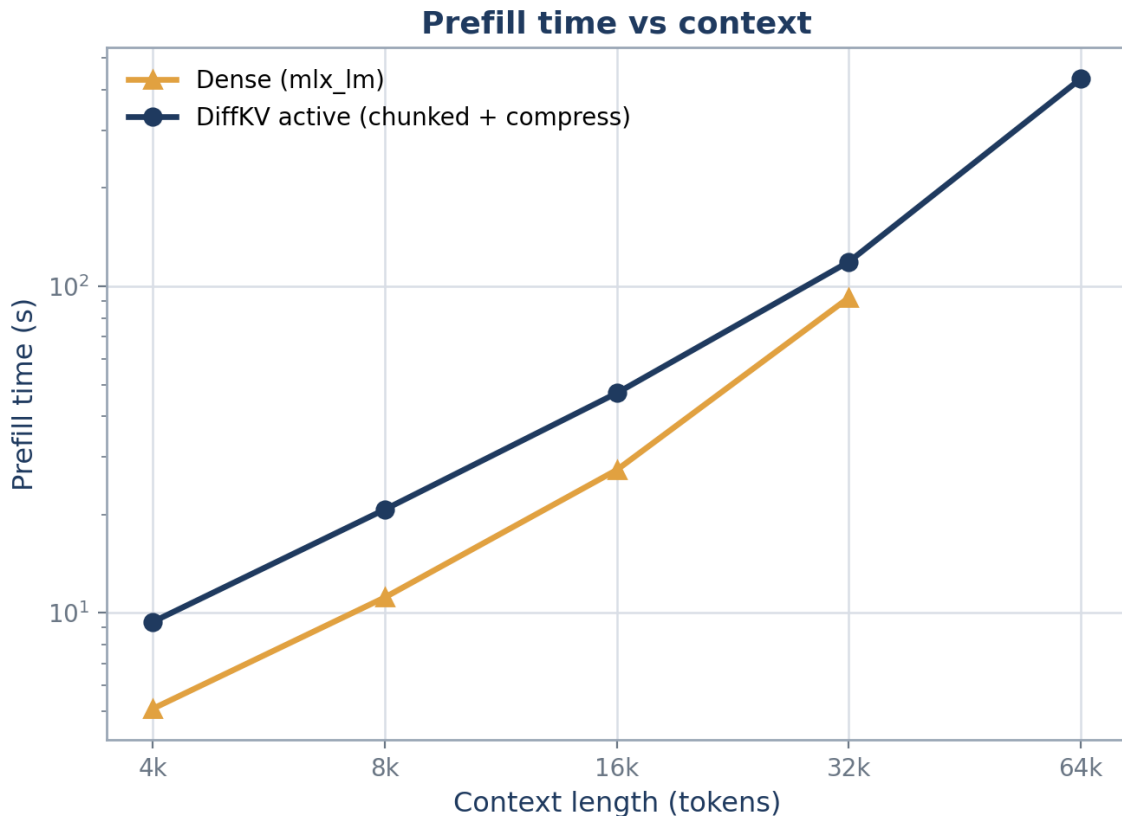


Figure 8: Prefill time vs. context (log scale). Exact attention plus streaming SVD and residual extraction; near-linear growth, slowest among the MLX configurations.

grows linearly in R and the compression ratio falls. The default $R=64$ sits at the exact-recall end of this curve; a deployment that tolerates approximate recall can trade down for a smaller store. This curve is the central design characterization of DIFFKV: it makes explicit that the runtime’s *exact* recall is bought with memory, and quantifies the price.

8.8 Allocator peak and the combined view

Figure 10 reports the MLX allocator peak, which is nearly identical across all three configurations at matched context because it is set during prefill; compression lowers the *decode-phase* resident memory and enables context reach, not this peak (§9). Figure 11 combines footprint, throughput, and prefill in one view.

9 Analysis and Discussion

9.1 What the compression actually buys

The cleanest statement of DIFFKV’s benefit is at the level of KV state, not allocator peak. The compressed store is a *pre-allocated, fixed* pool: its footprint is a constant of the configuration (≈ 0.68 GB across 28 layers) regardless of context, whereas the dense full-KV cache grows linearly

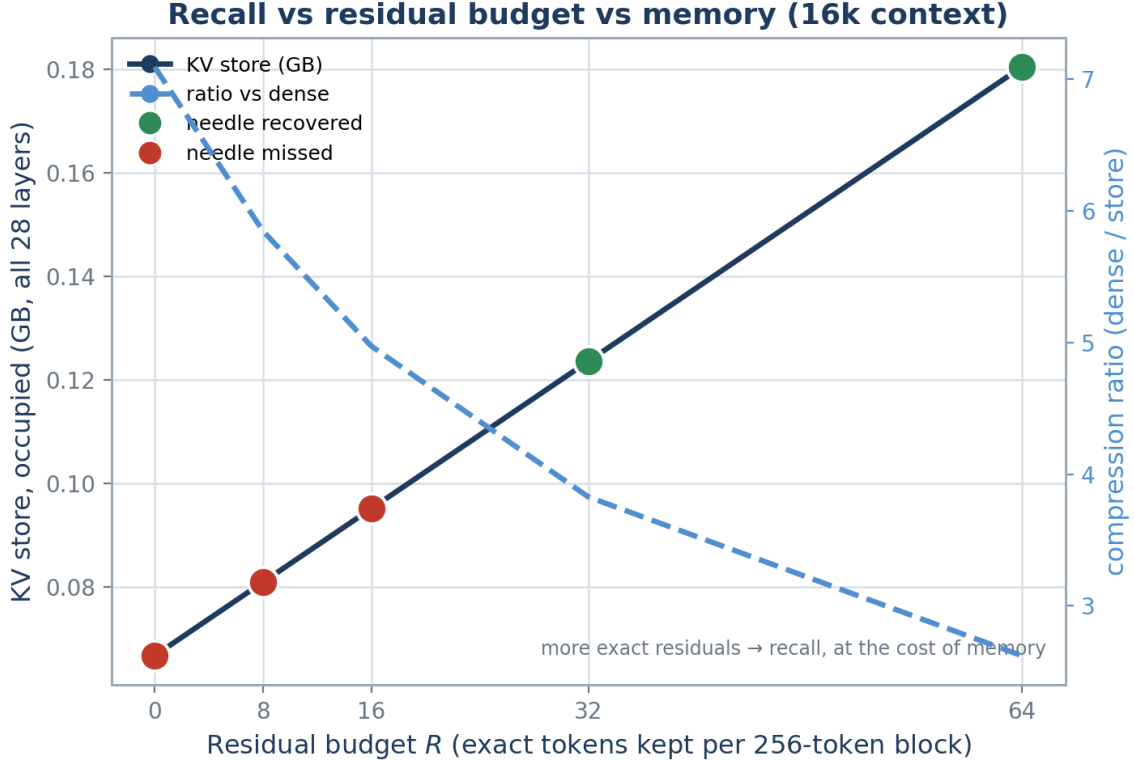


Figure 9: Recall vs. residual budget vs. memory at a fixed long context. Green/red markers are needle recovered/missed; the store grows with R while the compression ratio falls. Exact recall is bought with memory.

and crosses the store’s size early and keeps going (Fig. 6, Table 2). The reduction is $\approx 2.85\times$ per block at equal precision—a real but modest factor, because a quarter of every block is kept exact as residuals. Compression alone, then, is not the whole story; the system-level win is that a *bounded* store plus the cache-drop at the prefill $\rightarrow$ decode boundary lets the runtime address contexts the dense cache cannot fit on the same device (§8, E2). The r -vs- $H_{kv}d$ gap of §4 sets the low-rank part of that bound; the residual budget sets the rest.

9.2 Why the allocator peak tells a different story

The MLX allocator peak is nearly identical for the compressed runtime, the exact-decode ablation, and the dense baseline at matched context, because the peak is set during *prefill*—a full attention over the growing native cache that all three perform—and is dominated by weights and transient activations rather than by the retained KV state (Fig. 10). Compression reduces the *steady-state* decode footprint, not the prefill peak. We make this explicit rather than reporting only a process-level memory number that would credit compression with a peak reduction it does not deliver. The practical implication is that on a memory-constrained device the binding event is whichever is larger of (a) the prefill peak and (b) the decode-time resident set; DIFFKV lowers (b) and bounds it, but leaves (a) to prefill-side techniques (chunking, activation release) the runtime already applies. The reach result of E2 is precisely the regime where (b), accumulated over a long generation, is what would otherwise exhaust memory.

Table 3: Residual-budget trade-off (E6): recall, decode throughput, and store footprint as the exact-residual budget R varies, at a fixed long context.

R	Needle	Decode tok/s	Store (GB)	Ratio vs dense	Per-block (KiB)
0	✗	17.1	0.067	$7.09\times$	26.0
8	✗	14.9	0.081	$5.85\times$	34.0
16	✗	13.8	0.095	$4.97\times$	42.0
32	✓	10.4	0.124	$3.83\times$	58.0
64	✓	10.2	0.180	$2.62\times$	90.0

9.3 The throughput cost

The compressed decode is several times slower than dense decode and roughly flat across context (Fig. 7). The flatness follows from the compute profile of §6: with a fixed top- K , the low-rank aggregation and the exact-residual attention are context-independent, and only the router is linear in context with a small Rd constant. The absolute gap is an implementation property, not an algorithmic necessity: the kernel expresses many small tensor contractions inside one compiled graph, and on this hardware that does not reach the throughput of MLX’s native fused attention over a contiguous cache. The exact-decode ablation—identical store, attention over the retained native cache—recovers dense-like throughput, which localizes the gap to the compressed-kernel dispatch rather than to anything the compression or routing precludes. A hand-written fused decode (a custom Metal op, or CUDA/Triton on NVIDIA) is the natural way to close it, and the adaptive policy avoids paying it at all until context makes memory the binding cost.

9.4 Fidelity: restored by residuals, at a memory price

A pure low-rank block cannot reproduce a buried single-token detail once it leaves the recency window: the needle’s delta is a low-energy outlier that rank-16 truncation discards. DIFFKV resolves this not by raising the rank but by keeping the block’s highest-reconstruction-error tokens *exactly* (§4.4) and attending them at full precision (§6). With the default budget the compressed decode recovers the needle at every tested context (§8, E5)—the qualitative correctness the residual-free version lacked. The cost is memory: E6 (Fig. 9, Table 3) shows recall turning on as the budget grows while the store grows with it. The residual-key router (Eq. 8) is the second half of the fix—a low-rank or min/max *summary* score averages the outlier away and can drop the needle’s block from the top- K at large block counts, whereas scoring the exact residual keys keeps it. Both mechanisms are content- and model-agnostic: they act on whichever tokens a block finds distinctive, with no tuning to the needle.

Observation 1. DIFFKV trades *decode throughput and a modest amount of memory* (the residuals) for *bounded, context-independent KV state with exact retrieval*. The trade is favorable exactly when the KV cache is the binding constraint—long context on a memory-limited device—and the throughput cost is acceptable or removed by a fused kernel; it is unfavorable when the device has memory to spare, where the adaptive policy simply decodes exactly.

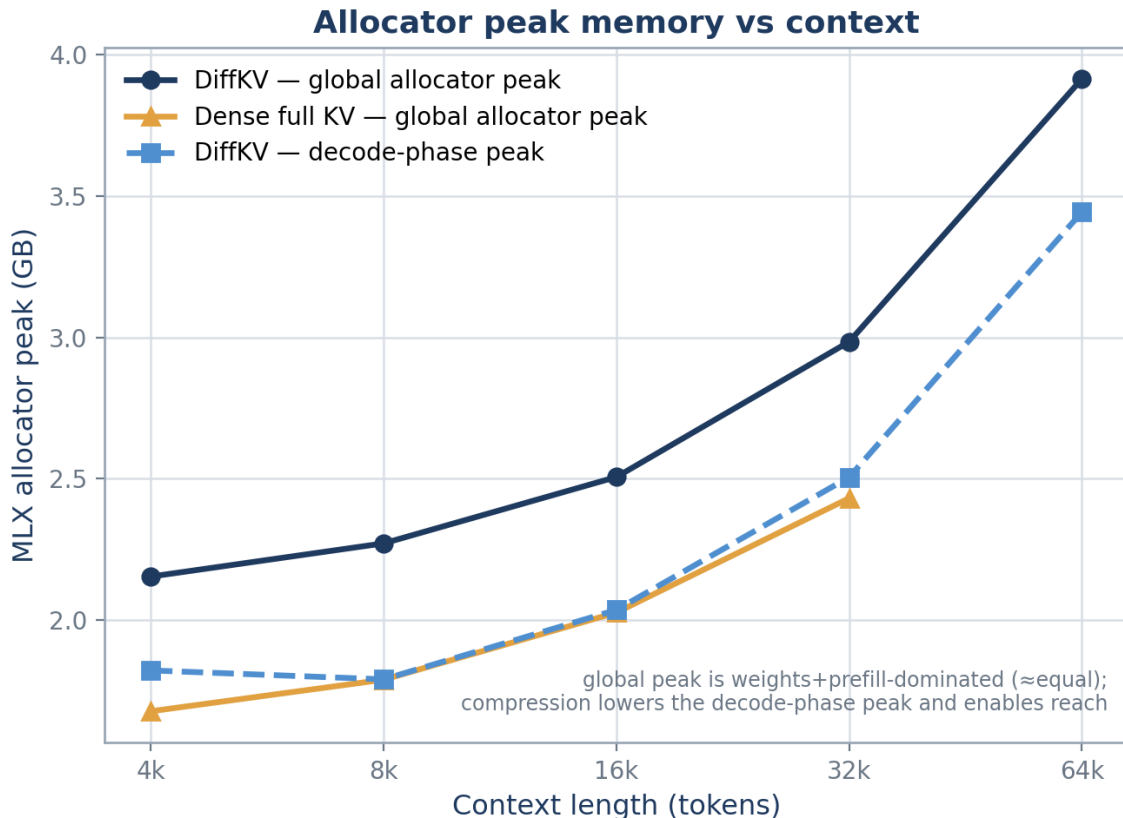


Figure 10: MLX allocator peak vs. context. Nearly identical across configurations—peak is weights- and prefill-dominated. Compression’s win is context reach and steady-state decode memory, not the global peak; we report it honestly rather than overstate a peak reduction.

10 Limitations and Future Work

Decode throughput (primary). The fused kernel is expressed as many small ops in a compiled graph and does not yet match native fused attention (§9); the exact-decode ablation localizes the gap to the kernel dispatch, not the algorithm. A hand-written fused decode—a custom Metal op, or a CUDA/Triton kernel on NVIDIA hardware—is the natural path to closing it. We have deliberately structured the throughput figures and the main results table so that a future **CUDA/Triton fused-decode** measurement can be added as one more series without restructuring the evaluation; no such number is reported or estimated here, and the placeholder is labelled as future work. Until then, the adaptive policy keeps the cost off the short-context path entirely.

Compression ratio and the residual memory ceiling. Exact recall is bought with memory: at the default budget a quarter of every block is stored verbatim, so the per-block reduction is $\approx 2.85\times$, not the order of magnitude a pure low-rank scheme suggests (§4). The residuals also set a scaling ceiling—at a million tokens the residual K/V alone would dominate the pool—so very long contexts need either a smaller budget (trading recall, per E6) or a two-stage router that gathers residuals only for selected blocks (which the runtime already does at decode, but the store still holds them). A budget that adapts to per-block content, rather than a fixed R , is the natural refinement.

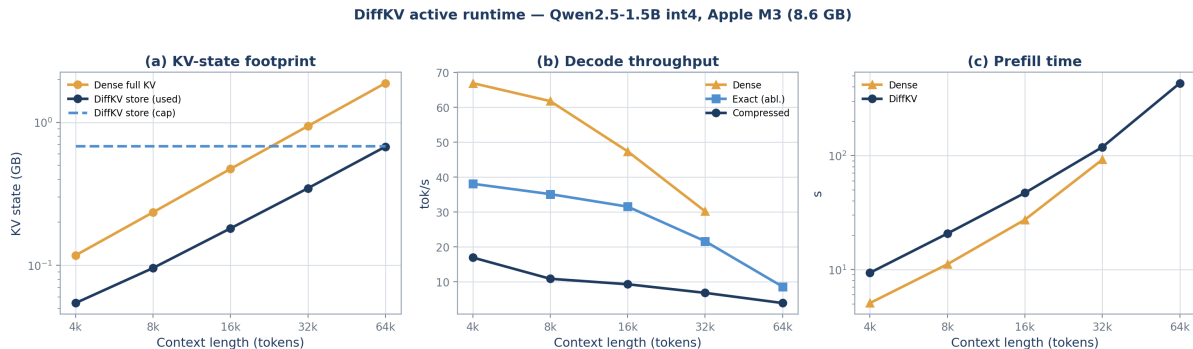


Figure 11: Combined view: (a) KV-state footprint, (b) decode throughput, (c) prefill time, for DIFFKV compressed, the exact ablation, and the dense baseline.

Router cost. The default residual-key router scores every block’s anchor and top residual keys each step, an $O(nRd)$ cost linear in context. It is cheap relative to the kernel at the scales we test, but at extreme context a cheap pre-filter feeding the exact rerank would be needed—taking care that the pre-filter cannot drop the needle’s block, the failure mode that motivated the exact router in the first place.

Prefill cost and peak. Prefill remains exact and therefore $O(N)$ in attention, with the per-block SVD and residual extraction added on top, making it the slowest phase and the dominant wall-time cost at large context. The global memory peak is also set during prefill (§9); DIFFKV does not reduce it. Compressed or sparse prefill is a known direction (prototyped in the repository) but is out of scope here.

Evaluation breadth. We report one model (Qwen2.5-1.5B int4), one device (Apple M3, 8.6 GB), and one probe (needle-in-a-haystack) with greedy decoding. Exact-string recall is a sharp but narrow correctness signal; broader claims require additional models and sizes, perplexity and downstream-quality metrics, multi-needle and varied-depth probes, and batched/multi-session throughput. The serving stack supports batching and multi-session residency; we did not exercise them here.

The grounding module. The semantic-retrieval/factual-binding layer is inactive in the benchmarked runtime (§7). Whether it improves grounded long-context QA on top of the compressed cache is an open, separately evaluable question.

Portability. The runtime is specialized to MLX and to the Qwen2 attention module it patches. A portable PyTorch/Triton backend and a C++/GGML reimplementations target the same design in the repository but are under active development and are not evaluated here.

11 Conclusion

DIFFKV treats the KV cache as a structured object rather than an opaque tensor: each block is a representative anchor, a low-rank correction, and a small set of exact residual tokens, and attention

is computed directly from that representation without ever rebuilding the dense cache. The low-rank term and a bounded pool give a KV state that is fixed and context-independent; the residuals give back the verbatim recall that pure low-rank compression loses. The payoff is that a small model addresses contexts that exhaust a full-KV baseline on the same 8 GB device, while recovering a buried passcode exactly at every context we test. We have been explicit about the costs measured on the way there: the compressed decode is slower than dense decode on current hardware—a kernel-dispatch gap we localize precisely with an exact-decode ablation on the identical store—the exact recall is bought with the memory the residuals occupy ($\approx 2.85\times$ reduction, not an order of magnitude), and the per-block SVD makes prefill the slowest phase. We report the compressed path as the primary result and the exact path only as an upper bound, attributing every number to the code that produced it.

The most valuable next steps are sharp and already scoped: a hand-written fused decode kernel to close the throughput gap, a content-adaptive residual budget to improve the compression ratio and push past the residual memory ceiling at extreme context, and a CUDA/Triton implementation whose results can drop into the existing figures unchanged. The design—anchor plus differential low-rank plus exact residuals, attention routed and computed in low-rank space, a bounded pool with a recency window and an adaptive decode policy—stands independent of those refinements, and the runtime that realizes it is small, composes with stock quantized models, and runs on the hardware people already have.

References

- [1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [2] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. arXiv preprint, 2023.
- [3] Tri Dao, Daniel Y Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [4] DeepSeek-AI. DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model. arXiv preprint, 2024. Introduces Multi-head Latent Attention (MLA), a low-rank latent KV cache.
- [5] Nathan Halko, Per-Gunnar Martinsson, and Joel A Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288, 2011.
- [6] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. MLX: Efficient and flexible machine learning on apple silicon. <https://github.com/ml-explore/mlx>, 2023.
- [7] Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023.
- [8] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large

- language model serving with PagedAttention. In *Symposium on Operating Systems Principles (SOSP)*, 2023.
- [9] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. arXiv preprint, 2024.
 - [10] Zichang Liu, Aditya Desai, Fangshuo Liao, Weitao Wang, Victor Xie, Zhaozhao Xu, Anastasios Kyrillidis, and Anshumali Shrivastava. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
 - [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhao Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. arXiv preprint, 2024.
 - [12] Jiaming Tang, Yilong Zhao, Kan Zhu, Guangxuan Xiao, Baris Kasikci, and Song Han. Quest: Query-aware sparsity for efficient long-context LLM inference. In *International Conference on Machine Learning (ICML)*, 2024.
 - [13] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024.
 - [14] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Reproducibility

All results were produced on a single machine, one engine per isolated subprocess, with every number measured (failed cells are reported as failures, never filled with estimates).

Hardware. Apple M3, 8.59 GB unified memory (host `0ms-MacBook-Pro-2.local`), macOS (Darwin 25.x). The device is shared with the OS and other applications, so absolute memory thresholds shift with machine load; the scaling slopes and engine ordering are the stable results.

Software. Python (`diffkv_venv`), MLX 0.31.2, `mlx_lm`, NumPy, PyTorch (used only for tensor plumbing in the wrapper, not for compute), `psutil`. Model: `mlx-community/Qwen2.5-1.5B-Instruct-4bit` (group size 64, 4-bit), 28 layers, 12 query heads, 2 KV heads, head dim 128, vocab 151,936, trained context 32,768, RoPE $\theta = 10^6$.

Configuration. `rank=16, block_size=256, recency_window=512` (dense buffer 768), `max_blocks=256, max_residual=64, top-K=16` routing with the `residual` router, `DIFFKV_SVD_SEED=1234` (deterministic), preset “mid”. The shipped decode policy is `DIFFKV_COMPRESSED_DECODE=auto` (compressed at/above `DIFFKV_COMPRESSED_MIN_CTX=16k`, else exact); for the ablation it is forced to 1 (compressed, primary) or 0 (exact, upper bound).

Protocol. A needle-in-a-haystack chat prompt is built once per context length from the Qwen2.5 tokenizer and fed verbatim (raw mode, no double chat-templating) to every configuration. The needle is a planted passcode (OMEGA-7741-DELTA); “needle recovered” means the generation reproduced it—a free correctness signal. Engines are warmed once (a 1-token forward to compile kernels) before timing. Prefill is timed with `perf_counter` around the chunked forward after `mx.eval`; decode is fixed-length greedy (128 tokens, EOS ignored) so throughput is directly comparable. Memory is read from MLX’s allocator (`get_peak_memory/get_active_memory`); the decode-phase peak resets the peak counter at the prefill→decode boundary to isolate steady-state decode memory. Analytic KV-state bytes are computed from the live store occupancy.

Commands. The full measured dataset is produced by one clean, sequential batch (run with nothing else competing for CPU: the per-block NumPy SVD during prefill is CPU-bound, and concurrent processes inflate prefill time):

```
# clean batch: decode-mode ablation (4k-32k) + 64k reach + residual sweep
zsh paper/scripts/run_paper_measurements.sh

# equivalently, the individual drivers it calls:
diffkv_venv/bin/python3 paper/scripts/measure_active.py \
-ctx 4096 8192 16384 32768 -modes compressed exact -gen 128 \
-out paper/generated/active_modes_sweep_v2.json
diffkv_venv/bin/python3 paper/scripts/measure_residual_sweep.py \
-ctx 16384 -residuals 0 8 16 32 64 -out paper/generated/residual_sweep.json

# dense full-KV baseline (same prompts, same memory metric)
diffkv_venv/bin/python3 benchmarks/run_bench.py -engines dense \
-contexts 4096 8192 16384 32768 -gen 128 -ram-cap-gb 7.5

# figures + tables from the measured JSON (no hand-typed numbers)
diffkv_venv/bin/python3 paper/scripts/make_diagrams.py
diffkv_venv/bin/python3 paper/scripts/make_graphs.py
diffkv_venv/bin/python3 paper/scripts/make_tables.py
```

Artifacts. Measured JSON: `paper/generated/active_modes_sweep_v2.json` (active ablation, both modes), `paper/generated/active_modes_sweep_64k.json` (64k reach), `paper/generated/residual_sweep.json` (residual-budget trade-off), and `benchmarks/results/PAPER_dense_sweep.json` (dense baseline). Figures are regenerated from these by the scripts above; tables are emitted to `paper/tables/`. The provenance, the decode-path attribution that motivated this measurement design, and the corrected compression-ratio accounting are documented in `paper/notes/`.

B Full Measured Results

Table 4 lists every measured cell from the instrumented sweep (`paper/scripts/measure_active.py`) and the dense baseline (`benchmarks/run_bench.py`). *Decode-phase peak* resets the MLX peak counter at the prefill→decode boundary to isolate steady-state decode memory; *Store* is the analytic occupied KV-state footprint of the compressed pool plus recency window; *Full KV* is the analytic dense-cache size for the same sequence length (all 28 layers, fp16). Empty cells are measurements that do not apply to that configuration; failed cells (e.g. a dense out-of-memory at 64k) are stated in the text, never given an invented value.

Table 4: Per-run detail: compressed decode (primary), exact decode (ablation), dense baseline.

Config	Ctx	Prefill (s)	Decode tok/s	MLX peak (GB)	Dec. peak (GB)	Store (GB)	Full KV (GB)	Need
compressed	4k	9.4	16.9	2.15	1.82	0.054	0.117	✓
compressed	8k	20.7	10.8	2.27	1.79	0.096	0.235	✓
compressed	16k	47.1	9.3	2.51	2.03	0.181	0.473	✓
compressed	32k	118.9	6.8	2.98	2.50	0.346	0.942	✓
compressed	64k	432.5	3.9	3.91	3.44	0.676	1.881	✓
exact	4k	9.6	38.0	2.15	1.75	0.054	0.117	✓
exact	8k	19.7	35.1	2.27	1.94	0.096	0.235	✓
exact	16k	45.9	31.5	2.51	2.04	0.181	0.473	✓
exact	32k	116.8	21.6	2.98	2.51	0.346	0.942	✓
exact	64k	449.6	8.6	3.91	3.45	0.676	1.881	✓
dense	4k	5.1	66.9	1.68	—	—	—	✓
dense	8k	11.1	61.7	1.79	—	—	—	✓
dense	16k	27.4	47.3	2.03	—	—	—	✓
dense	32k	92.3	30.1	2.43	—	—	—	✓

C Notation

Symbol	Meaning
L	transformer layers (28)
H	query heads (12)
H_{kv}	key/value heads (2)
$G = H/H_{kv}$	GQA group size (6)
d	head dimension (128)
N	sequence length (tokens)
B	block size (256)
$S = B - 1$	compressible tokens per block (255)
r	SVD target rank (16; adaptive in $[4, r]$)
R	exact residual budget per block (64)
K	top- K blocks routed at decode (16)
W	dense recency window (512; buffer $W+B = 768$)
M	compressed-pool capacity (256 blocks)
a_k, a_v	anchor key / value (token 0 of a block)
U	per-token coefficients $\mathbb{R}^{S \times r}$ (shared K/V)
V_K, V_V	key / value bases $\mathbb{R}^{H_{kv} \times r \times d}$
R_K, R_V	exact residual keys / values $\mathbb{R}^{R \times H_{kv} \times d}$
$k^{\min}, k^{\max}$	per-block element-wise key min / max (router)
ρ_b	router relevance of block b (Eq. 8)
s	SVD normalization scale (fp32)
q, σ	query, attention scale
$\ell^{\text{sp}}, \ell^{\text{dn}}$	log-sum-exp of low-rank / exact branch